

Additionally, we experimented with enabling trainable biases in LLaMA’s layernorms and switching from *silu* to *gelu* (to more closely resemble GPT2), in a similar way as Figure 5, but found these changes do not affect the model’s capacities. We ignore those experiments for clarity.

Below, we discuss our parameter choices for the experiments in Figure 4 and Figure 5.

Parameter 7 (Figure 4). In the 100-exposure setting,

(a) For LLaMA/Mistral models on bioS(N) data, aiming to present *negative* results, we select the best learning rate from three options in each data setting:

- For $N = 50K$, we use $wd = 0.01$, $lr = 0.0003/0.0005/0.001$, and batch size 12 with bf16;
- For $N = 100K$, we use $wd = 0.01$, $lr = 0.0003/0.0005/0.001$, and batch size 24 with bf16;
- For $N = 200K$, we use $wd = 0.01$, $lr = 0.0003/0.0005/0.001$, and batch size 48 with bf16;
- For $N = 500K$, we use $wd = 0.01$, $lr = 0.0002/0.0003/0.0005$, and batch size 96 with bf16;
- For $N = 1M$, we use $wd = 0.01$, $lr = 0.0002/0.0003/0.0005$, and batch size 192 with bf16;
- For $N = 2M$, we use $wd = 0.01$, $lr = 0.0003/0.0005/0.001$, and batch size 384 with bf16;
- For $N = 5M$, we use $wd = 0.01$, $lr = 0.0003/0.0005/0.001$, and batch size 768 with bf16;
- For $N = 10M$, we use $wd = 0.01$, $lr = 0.0003/0.0005/0.001$, and batch size 1536 with bf16;
- For $N = 20M$, we use $wd = 0.002$, $lr = 0.0003/0.0005/0.001$, and batch size 1536 with bf16.

(For $N \leq 1M$, we also tested the same settings with fp16, finding similar results. However, LLaMA/Mistral models tend to fail more often with fp16, so we primarily used bf16.)

(b) For GPT2_{1/4}:

- For $N = 50K$, we use $wd = 0.01$, $lr = 0.0005/0.001$, and batch size 12 with fp16;
- For $N = 100K$, we use $wd = 0.01$, $lr = 0.0005/0.001$, and batch size 24 with fp16;
- For $N = 200K$, we use $wd = 0.01$, $lr = 0.0005/0.001$, and batch size 48 with fp16;
- For $N = 500K$, we use $wd = 0.01$, $lr = 0.0003/0.0005$, and batch size 96 with fp16;
- For $N = 1M$, we use $wd = 0.01$, $lr = 0.0003/0.0005$, and batch size 192 with fp16.

(c) For GPT2₀, to present a *negative* result, we use the same settings as in Parameter 4(a):

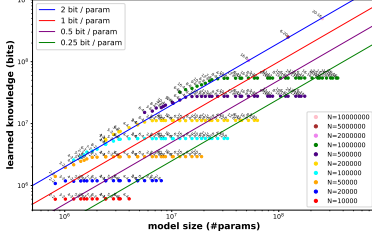
- For $N = 50K$, we use $wd = 0.01$, $lr = 0.0003/0.0005/0.001$, and batch size 12 with bf16;
- For $N = 100K$, we use $wd = 0.01$, $lr = 0.0003/0.0005/0.001$, and batch size 24 with bf16;
- For $N = 200K$, we use $wd = 0.01$, $lr = 0.0003/0.0005/0.001$, and batch size 48 with bf16;
- For $N = 500K$, we use $wd = 0.01$, $lr = 0.0002/0.0003/0.0005$, and batch size 96 with bf16;
- For $N = 1M$, we use $wd = 0.01$, $lr = 0.0002/0.0003/0.0005$, and batch size 192 with bf16.

Parameter 8 (Figure 5). In the 100-exposure controlled comparison experiment,

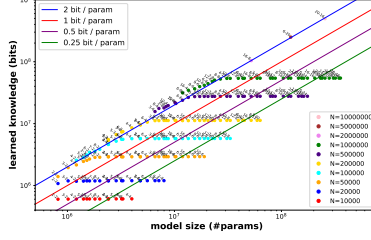
- For presenting *negative* results (Figure 5(a) and Figure 5(c)), we select the best learning rate from three options, identical to GPT2₀ in Parameter 4(c).
- For presenting *positive* results (Figure 5(b) and Figure 5(d)), we use a single set of learning rates, identical to Parameter 2 but with fp16 replaced by bf16 for a stronger comparison.

C More on Quantization

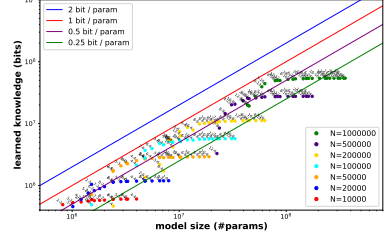
We use the `auto_gptq` package (based on [10]) to quantize the GPT2 model results in Figure 1 for the bioS data and the GPT2 model results in Figure 2 for the bioD data. We simply use a small set of 1000 people’s biographies to perform the quantization task. Our results are presented in Figure 14 for the bioS data and in Figure 15 for the bioD data.



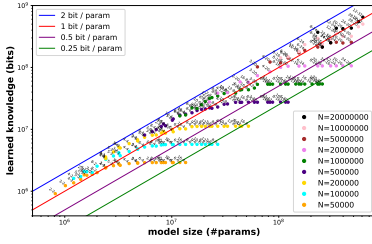
(a) Same Figure 1(a), **1000 exposures**



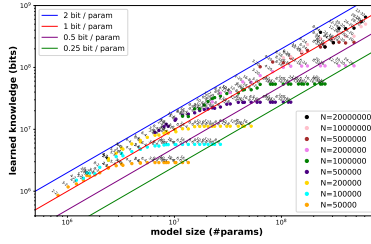
(b) Figure 1(a) quantized to 8bit



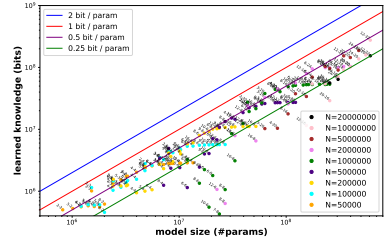
(c) Figure 1(a) quantized to 4bit



(d) Same Figure 1(b), **100 exposures**



(e) Figure 1(b) quantized to 8bit



(f) Figure 1(b) quantized to 4bit

Figure 14: Scaling laws for GPT2 after quantizing Figure 1 into int8 and int4.

Conclusion. Quantizing a mixed-precision fp16 trained model into int8 shows no change, but quantizing into int4 results in a capacity ratio loss greater than 2x.

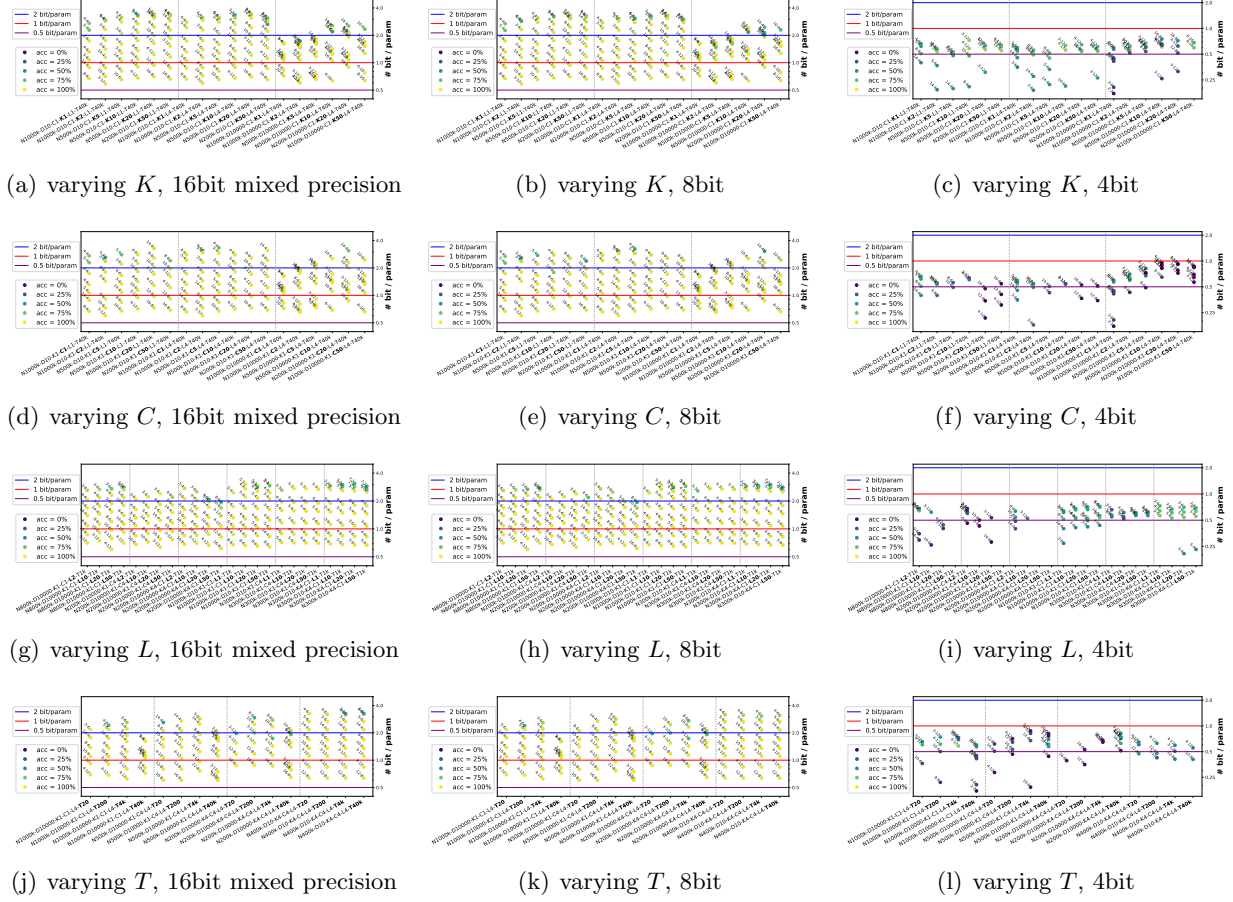


Figure 15: 8-bit/4-bit quantization of GPT2 models trained on $\text{bioD}(N, K, C, D, L, T)$ data for **1000 exposures**. **Left:** Identical to Figure 2, showing only models with accuracy $\geq 50\%$; **Middle:** After quantization to 8-bit; **Right:** After quantization to 4-bit, including models with all accuracies.

Observation: For the bioD data family, quantizing to 8-bit has negligible impact on model capacities. Quantizing to 4-bit reduces capacity by more than 2x, especially for large D and L , leading to significantly larger reductions and explaining the missing columns in Figure 15(i)).

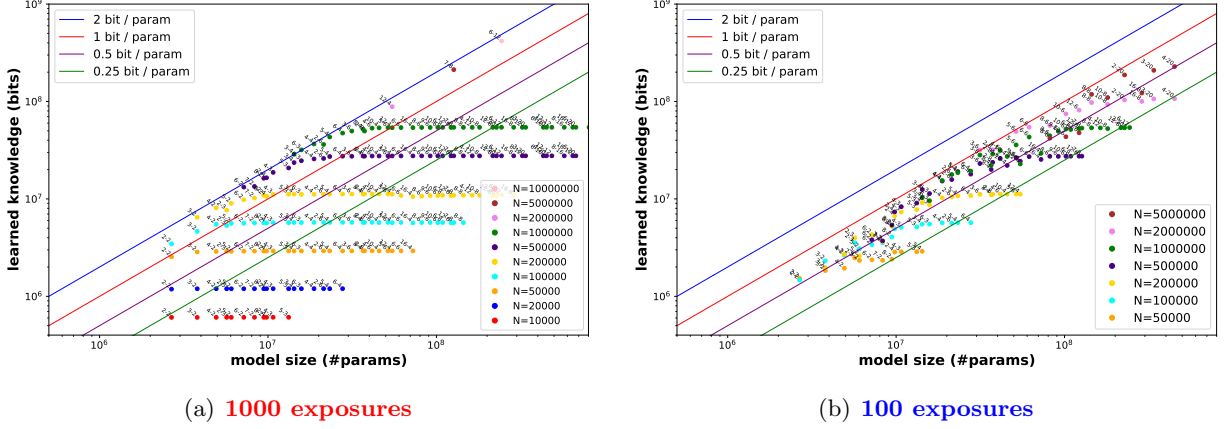


Figure 16: Scaling laws for the mixture-of-experts GPT2 models with 32 experts on the bioS(N) data.

Remarks. Compared to Figure 1, using MoE with 32 experts reduces the 1000-exposure capacity by 1.3x and the 100-exposure one by 1.5x, despite the MoE model using 11.3x fewer parameters during inference. For the strongest result we use $topk = 1, cap_factor = 2$; other variants are in Figure 17.

D More on Mixture of Experts

We utilize the `tutel` package for implementing Mixture-of-Experts (MoE) on GPT2 models [18]. In MoE, the parameter $topk$ determines the number of experts each token is routed to. It is recommended by some practitioners to use $topk = 2$ during training and $topk = 1$ during testing. Additionally, the cap_factor parameter ensures that, given M experts, each expert receives no more than $\frac{cap_factor}{M}$ fraction of the data.

Using $topk = 1$ and $cap_factor = 1$ is generally not advisable. Thus, to provide the strongest result, we set $topk = 1, cap_factor = 2$ for the 1000/100-exposure scaling laws in Figure 16. (During testing, we increase the capacity factor to $cap_factor = 8$.)

For the 100-exposure scaling law, we additionally compare three configurations: $(topk, cap_factor) = (1, 2), (2, 1), (2, 2)$, finding minimal differences among them as shown in Figure 17. Remember from Section 7 that differences in model architecture usually become apparent in the insufficient training regime; this is why we opt for 100-exposure instead of 1000-exposure. Notably, $(topk, cap_factor) = (2, 2)$ performs best (among the three) for deep models, such as GPT2-16-4 with 32 experts.

Due to their sparsity, MoE models often require higher learning rates compared to dense models. Consequently, we adjust the optimizer parameters as follows:

Parameter 9 (Figure 16, Figure 17). In the 1000-exposure setting for GPT2-MoE models with 32 experts, we slightly increase the learning rates while keeping other parameters nearly identical to Parameter 1:

- For $N = 10K$, we use $wd = 0.02$, $lr = 0.001/0.002$, and batch size 24 with fp16;
- For $N = 20K$, we use $wd = 0.02$, $lr = 0.001/0.002$, and batch size 48 with fp16;
- For $N = 50K$, we use $wd = 0.02$, $lr = 0.001/0.002$, and batch size 96 with fp16;
- For $N = 100K, 200K$, we use $wd = 0.02$, $lr = 0.001/0.002$, batch size 192 with fp16;
- For $N = 500K, 1M$, we use $wd = 0.01$, $lr = 0.0005/0.001$, batch size 192 with fp16;
- For $N = 2M$, we use $wd = 0.005$, $lr = 0.002$, and batch size 1536 with fp16;
- For $N = 5M$, we use $wd = 0.002$, $lr = 0.0005$, and batch size 1536 with fp16;
- For $N = 10M$, we use $wd = 0.001$, $lr = 0.0005$, and batch size 1536 with fp16.

In the 100-exposure setting, we also use higher learning rates compared to Parameter 2:

- For $N = 50K$, we use $wd = 0.01$, $lr = 0.001/0.002/0.005$, and batch size 12 with fp16;